PAGE 1 — SQL/RA · STORAGE · INDEXING · SORT/HASH/JOINS · OPTIMISATION

SQL & RELATIONAL ALGEBRA

SQL Evaluation Order

FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.

RA Operators

- **σ** selection (rows), **π** projection (cols), **ρ** rename.
- **∪** union, **−** difference, **×** cross product, ∩ = R−(R−S).
- ⋈ join = $\sigma_\theta(R \times S)$; natural join projects away duplicate cols.
- Theta join ⊃ Equi-join ⊃ Natural join.
- **Left anti-join**: tuples in R with no match in S. **Outer joins**: pad unmatched side with NULL.

Equivalences (rewriting heuristics)

- Push σ down (filter early); push π down (drop attrs early).
- σ cascade: $\sigma_{p \wedge q} = \sigma_p(\sigma_q)$; σ commutes; π cascade.
- ⋈ assoc/commut; avoid × (replace with ⋈ + condition).

Codd's Theorem

Relational Calculus ≡ Relational Algebra in expressiveness.

STORAGE & DISK

Hierarchy

CPU Reg (<1ns) → Cache (<10ns) → DRAM (100ns) → SSD (0.1ms) → HDD (10ms) → Network (30ms).

Disk Time

`T = Seek + Rot. delay + Transfer`

Seq I/O ≫ random (≈100× HDD). Outer tracks have more sectors at same RPM ⇒ **higher transfer rate on outer cylinders**. Place hot small files on outer tracks, large sequential files in contiguous cylinders.

Page = Block

Unit of I/O, 4–64 KB. Disk-space mgr maps pageId → physical loc.

Prefetching

Reads ahead into pool while CPU works on current page → hides I/O latency on sequential scans. DBMS may prefer its own prefetch (knows access pattern, supports asynchronous I/O, can avoid polluting buffer with read-once pages).

BUFFER MANAGEMENT

Pool of frames; each holds `pageId`, dirty flag, **pin counter** (not flag — many readers share a page).

API

`pin(pid)`: in pool → pinCnt++; else evict victim (write back if dirty), read in. `unpin(pid, dirty)`: pinCnt−−; OR-in dirty. Also `newPage`, `freePage`.

Replacement Policies

- **LRU**: evict least-recently *unpinned*. Bad for sequential flooding (scan with N+1 pages → all misses).
- **Clock**: approx LRU; ref bit + circular sweep. Skip pinned; ref=1→clear; ref=0→evict.
- **MRU**: better for repeated full scans.

Why pinCount, not pinFlag? multiple operators may need the same page; flag would lose count. **Why don't evict on pinCnt=0?** Because that page may still be the cheapest *kept*; replacement chooses based on policy, not first-fit. **All dirty?** Pick a victim per policy, flush its dirty page (WAL-logs first), then load.

FILES, PAGES, RECORDS

File Organisations

- **Heap**: unordered (linked-list of pages or page directory). Fast insert (2 I/O), slow search.
- **Sorted**: keys ordered. Binary search $\log_2 P$. Insert costs ($\log_2 P$ + P).
- **Index**: B+ tree (range), hash (equality only).

Page Layout

- **Fixed-length packed**: `RID = (pid, slot#)`. Delete moves last record into the gap → **RID changes**. No good if external pointers exist.
- **Fixed-length unpacked**: bitmap of free slots; deletion just clears a bit. RIDs stable.
- **Slotted page**: header + slot directory of (offset, length); records grow from end of page toward middle. Variable-length OK; delete sets slot=−1 (RID stable). On overflow, may compact in place.

Slotted vs Packed (exam)

+ stable RIDs, variable-length records, in-place compaction.
− per-record indirection (slot dir lookup), tiny space overhead per slot.

Record Layout

Fixed: direct field offset. Variable: array of (offset,len) + NULL bitmap.

STORAGE MODELS & COMPRESSION

Model	Use	Layout
Row (NSM)	OLTP	All attrs of tuple together
Col (DSM)	OLAP	One attr contiguous; reconstruct via RID
PAX	Hybrid	Row groups → col chunks inside page

OLTP: short, point R/W, high concurrency. **OLAP**: scans, aggregates, read-heavy.

Columnar Compression

- **RLE**: (value, run-len). Best for sorted low-cardinality.
- **Delta**: encode differences; great for sorted numerics/timestamps.
- **Bit-packing**: use $\lceil \log_2(\max) \rceil$ bits per value.
- **Dictionary**: replace values with small codes; most common; supports predicates without decompressing.

FILE-ORG I/O COSTS

P = pages, R = recs/page, D = avg I/O time.

Op	Heap	Sorted
Scan	P·D	P·D
Eq. search (key)	0.5P·D	$\log_2 P \cdot D$
Eq. search (non-key)	P·D	($\log_2 P$+match)·D
Range search (M recs)	P·D	($\log_2 P$ + M/R)·D
Insert	2D	($\log_2 P$ + P)·D
Delete	(0.5P+1)·D	($\log_2 P$ + P)·D

Choose file org (exam rubric).

1. Daily-rebuild + bulk insert + 1 lookup: **heap** (insert is dominant).
2. Many range queries, few inserts: **sorted** or **clustered B+**.
3. Equality only: **hash**.
4. Many lookups + occasional insert: **clustered B+**.

INDEXES

Index Entries (k*)

A: full record stored in leaf (≤1 such index/table). B: ⟨k, rid⟩. C: ⟨k, [rid list]⟩.

Clustered vs Unclustered

Clustered ≈ data file order matches index, ≤1 per table. **Clustered range I/O ≈ #matching *pages***; **unclustered ≈ #matching *records*** (one I/O per RID, worst case).

ISAM

Static tree, overflow chains on insert. Fanout F≫2 → search $\log_F(N)$. Good for static data (less locking on inner).

B+ Tree

- Order d: every node has d ≤ #entries ≤ 2d (root may have ≥1).
- Inner: 2d keys + 2d+1 pointers; leaves: 2d entries + sibling ptr.
- Always balanced; height = #edges root→leaf.
- Typical fanout F ≈ 133, fill ~67%. h=3 → ~2.4M; h=4 → ~313M records.
- Optimisations: prefix compression (inner keys), suffix truncation.

B+ insert (split rule).

1. Descend to leaf L for key k.
2. If L has space, insert in sorted order. Done.
3. If full, split L into L (left) and L' (right). Per exam convention "new right gets larger half": L keeps d entries, L' gets d+1.
4. **Copy** L'.firstKey up to parent (leaf split = copy-up).
5. If parent full, split parent: **push** middle key up (inner split = push-up, key removed from inner).
6. Propagate; if root splits, new root with 1 key → height++.

B+ height bounds (insert 1..N, order d).

- h=0: ≤ 2d entries (single leaf).
- h=1: 2 ≤ #leaves ≤ 2d+1; entries ∈ [d+1, (2d+1)·2d].
- h=2: smallest N = (d+1)·d+1 leaves filled to d (min) is too few — for "minimum N to *force* h=2" use #entries that overflow root at h=1:
  $N_{min,h=2} = 2d \cdot (2d+1)+1$
- Largest N at h=2: full tree = $(2d+1)^2 \cdot 2d$ entries.
- Min entries in B+ of height h ≥1:
  $(d+1)^h \cdot d$
  (root 1 key, every inner d+1 children, every leaf d entries).

Hash Indexes

- Equality only. h(k) → bucket + chain.
- **Static**: chain grows on overflow → degrades.
- **Extendible**: directory size 2^g (global depth); each bucket has local depth d. Overflow: split bucket; if d=g, double directory.
- **Linear hashing**: split next bucket in round even if not the one overflowing → no directory.

EXTERNAL SORTING

N pages, B buffer pages.

```
Pass 0: ⌈N/B⌉ runs of B pages each (sorted in memory)
Passes 1+: merge B−1 runs at a time → fewer, longer runs
# passes = 1 + ⌈log_{B−1} ⌈N/B⌉⌉
Total I/O = 2·N · #passes
```

Blocked I/O: read/write b pages at a time → fewer seeks but each pass merges ⌊(B−b)/b⌋ runs. Reduces I/O *time*, may add a pass.

2020 example: N=9600, B=192 → Pass0: 50 runs of 192. Then $\lceil \log_{191} 50 \rceil$=1 pass. Total passes=2, I/O = 2·9600·2 = 38400 page-I/Os. Time = 38400·(10+5+1)ms.

Sort via clustered B+: traverse leaves — strictly better. Unclustered: 1 I/O per record — almost always worse than ext sort.

EXTERNAL HASHING & AGGREGATION

Phase 1 Partition: hash h_p → B−1 partitions, each ≤ B pages ideally.
Phase 2 Rehash: for each partition, build in-mem hash with $h_r \neq h_p$; emit groups/dups.

Need **B > √(f·N)** (f = fudge factor for in-mem table). Max table B·(B−1) pages.
Use **hash** for DISTINCT/GROUP BY (scales with #distinct).
Use **sort** if output ordering needed.

JOIN ALGORITHMS

R: M pages / m tuples. S: N pages / n tuples. B buffer pages.

Algorithm	Cost
Simple NLJ (per-tuple)	M + m·N
Page NLJ	M + M·N
Block NLJ (chunks of B−2)	M + ⌈M/(B−2)⌉·N
Index NLJ	M + m·(lookup-cost)
Sort-Merge	sort(R)+sort(S)+M+N
SMJ refined (merge during last pass)	3(M+N) when both fit in B^2
Grace Hash (GHJ)	3(M+N)
Build relation fits in B−2	M+N (optimal)

Choose join (exam recipe).

1. Equi-join & one fits in B → **in-mem hash/build** = M+N.
2. Equi-join, neither fits, both ≤ B^2:
   - Pre-sorted on key → **SMJ refined** = 3(M+N).
   - Otherwise → **Grace Hash** = 3(M+N).
3. Equi-join, index on inner → **Index NLJ** if outer small.
4. Theta/inequality → **Block NLJ** = M + ⌈M/(B−2)⌉N (smaller as outer).
5. Output needs sort → favour SMJ to avoid resort.

BNLJ Tips

Smaller as outer. Reserve 1 page for inner scan, 1 for output, B−2 for outer block. **Joining 3 rels at once**: outer P pages, two inner each ⌈(N−P)/2⌉ scanned for every outer chunk: cost ≈ N + ⌈N/P⌉·N + ⌈N/P⌉·⌈P/(B−1−P)⌉·N (full inner triple loop). Optimum P trades scans of two inners against rescans of outer.

Sort-Merge

Sort R, S on join key, then walk pointers. Refinement: if Pass-0 runs total ≤ B−1, last merge can also do the join → `3(M+N)`

.

Grace Hash

Phase 1: hash R on h_p; hash S on same h_p (cost 2(M+N) read+write). Phase 2: per-bucket build (smaller side) + probe (cost M+N). If a bucket overflows, recursive partition with h_r.

Outer / Anti-Join Adapts

BNLJ left anti: for each R tuple track "matched" flag; output only unmatched. Same cost as BNLJ. **SMJ left anti**: merge; if R-key < S-key advance R and emit; if equal skip all R-equal & advance both; if >, advance S. **Hash full outer**: in build, mark matched bucket entries during probe → at end emit unmatched build entries with NULLs; for probe side emit unmatched probe with NULLs as you go.

PROCESSING MODELS

Model	Dir	Emits	Use
Iterator (Volcano)	Top-down	1 tuple	General
Vectorised	Top-down	Batch	OLAP
Materialisation	Bottom-up	All	OLTP

Iterator: `open/next/close`; pipelined. Blocking ops (sort, hash-build, group-by) break pipeline. **Materialisation costing**: each non-leaf op writes out + next op reads in → add output size to cost.

QUERY OPTIMISATION PIPELINE

SQL → Parser → Rewriter (RA equiv.) → Optimiser (cost) → Plan → Executor.

Heuristic Rewrites

- Push σ down past ⋈ when predicate uses one side only.
- Push π down (drop unused attrs early).
- Convert × + σ to ⋈.
- Decompose conjunctions: $\sigma_{p \wedge q} \rightarrow \sigma_p(\sigma_q)$ (separately optimisable).

Access Methods

- **Sequential scan**: M I/O.
- **Index scan**: composite (A_1..A[?]) — matches m equality + ≤1 range on next attr (prefix-match rule).
- **Index-only scan**: query attrs ⊆ index → no heap I/O.
- **Multi-index / BitmapAnd / BitmapOr**: take RID sets from several indexes, intersect/union, fetch heap pages.

EXPLAIN Reading (PG)

Bitmap Heap Scan + BitmapAnd → two indexes (B+ on each cond, must support range). Index-Only Scan → covering index (B+ tree). Hash Join builds hash on inner. Seq Scan → no useful index or full table.

COST ESTIMATION & SELECTIVITY

sel(P) = |out|/|in| ∈ [0,1].

Predicate	sel()
A = c	1/NKeys(A)
A > a (uniform int)	(High−a)/(High−Low+1)
A IN (c_1..c[?])	k/NKeys(A)
A.x = B.y (join)	1/max{NKeys(A.x), NKeys(B.y)}
NOT P	1 − sel(P)
$P_1 \wedge P_2$	sel(P_1)·sel(P_2) (indep.)
$P_1 \vee P_2$	sel(P_1)+sel(P_2)−sel(P_1)·sel(P_2)

`|R ⋈ S| ≈ |R|·|S| / max{NKeys(R.A), NKeys(S.A)}`

Key–FK join (FK non-null): |R ⋈ S| = |many-side|.

Histograms

Equi-width (fixed bucket width) vs **equi-depth** (fixed tuples/bucket — better skew handling). For predicate A op a, identify bucket(s); within a bucket assume uniform.

Plan-cost recipe (estimating output size at each node).

1. Start at leaves: |scan(R)| = |R|, pages = $\lceil |R|/R_{pp} \rceil$.
2. σ: |out| = sel(P) · |in|. Re-derive #pages from new tuple count (or keep $|R|/R_{pp}$ if pages aren't recomputed).
3. ⋈: use formula above. With FK: |out| = |child rel|.
4. γ (group-by on A): |out| ≈ NKeys(A).
5. Apply materialisation: each non-scan op writes $|out|/R_{pp}$ pages and the next op reads them.

NOT Trick

NOT((A>5 ∧ A≤20) ∨ (A>10)) = ¬(A>5 ∧ A≤20) ∧ ¬(A>10) = (A≤5 ∨ A>20) ∧ (A≤10) = A≤5. Always simplify before reading histogram.

SEARCH STRATEGY (SYSTEM R)

- **Left-deep only**: inner is base relation → enables pipelining + index NLJ.
- **DP**: pass k builds best k-relation plans by extending best (k−1)-plans by 1 relation.
- Eliminate cross products. Keep cheapest plan per "interesting order" (ORDER BY, GROUP BY, future join keys).

Counting plans (n-relation join).

- Total left-deep orderings: **n!**.
- Total binary trees on n leaves (incl. non-left-deep): **n! ·C_{n-1}** where C_k=Catalan = (2k)!/((k+1)!k!).
- System R prunes: only orderings that avoid × (i.e., next relation must share a join attr with previous result). For chain query R-S-T-U: 2·(n−1)·… worked-example: 2020-Q1b had 4 rels, chain → System R considers **8 left-deep orders** (start at R or U, at each step extend with one neighbour).

PLAN-SPACE WORKED EXAMPLE

Sailors 500 pg / 80 tup; Reserves 1000 pg / 100 tup; 100 boats, 10 ratings. B=5.

Plan	I/Os
Page NLJ (no pushdown)	500 500
+ σ pushdown left (rating>5)	250 500
+ Reserves outer	6 000
+ Materialise inner	4 250
SMJ	3 540
Block NLJ + materialise	2 350
+ π pushdown	1 500
Index NLJ (cl. bid, uncl. sid)	1 010

TRANSACTIONS & ACID

- **Atomicity**: all-or-nothing (logging).
- **Consistency**: app + integrity constraints.
- **Isolation**: concurrent ≡ serial (CC).
- **Durability**: commits survive crashes (WAL).

Schedule S = interleaving of ops preserving per-txn order. Two ops **conflict** iff same object & ≥1 is W.

Conflict	Anomaly
R–W	Unrepeatable read
W–R	Dirty read
W–W	Lost update

SERIALIZABILITY

Conflict-equiv: same conflict pairs in same order. **Conflict-serializable** ⟺ ∃ serial conflict-equivalent.

Precedence-graph recipe.

1. Node per txn.
2. For each pair of conflicting ops (T_i first, T_j later): add edge $T_i \to T_j$ (don't duplicate).
3. **Conflict-serializable ⟺ acyclic.**
4. If acyclic, every **topological order** is a conflict-equivalent serial schedule. # of such serial schedules = # of topological orderings of the DAG.
5. If cyclic, remove one op from a cycle to break it; verify by rebuilding graph.

View-serializable: same initial reads, read-from, final writes. View ⊃ Conflict (allows blind writes). NP-hard; not used. **Serial ⊂ Conflict ⊂ View ⊂ All**.

LOCKING & 2PL

	S	X
S	✓	✗
X	✗	✗

2PL

Growing (acquire) → **Shrinking** (release). No new lock after first release. **2PL ⇒ conflict-serializable** (but not vice versa).

Strict 2PL

Hold ALL locks until commit/abort. Avoids cascading aborts & dirty reads. Standard in practice.

Deadlocks

Wait-for graph (detection).

1. Node per active txn.
2. If T_i waits on a lock held by T_j: edge $T_i \to T_j$.
3. Cycle ⇒ deadlock. Pick victim (least-work-done / youngest) and abort.

Prevention by timestamps (older = higher priority; restarted txn keeps original ts to prevent starvation).

Strategy	Old wants lock held by Young	Young wants lock held by Old
Wait-Die	Old waits	Young aborts (dies)
Wound-Wait	Old preempts (wounds Young)	Young waits

Both **avoid** all deadlocks (no cycle possible) but **do abort txns unnecessarily** (false positives — txns that wouldn't have deadlocked).

RECOVERY — WAL & ARIES

	STEAL	NO-STEAL
FORCE	Undo+No-Redo	No-Undo+No-Redo (simplest)
NO-FORCE	Undo+Redo (fastest)	No-Undo+Redo

Standard: **STEAL + NO-FORCE** → must do undo + redo via WAL.

WAL Rules

- **Rule 1**: log records for a page flushed before that page → enables UNDO (atomicity).
- **Rule 2**: all log records up to commit flushed before commit returns → enables REDO (durability).

Log Record Types

`BEGIN`, `UPDATE(LSN, txn, pid, before, after, prevLSN)`, `COMMIT`, `ABORT`, `END`, `CLR(undoNextLSN)`, `BEGIN_CHECKPOINT`, `END_CHECKPOINT(ATT, DPT)`.

Tables

ATT (active txn): {txn → lastLSN, status}. **DPT** (dirty page): {pid → recLSN = first LSN that dirtied it since last flush}.

ARIES — 3 phases.

1. **Analysis** (forward from last `BEGIN_CHECKPOINT`): rebuild ATT & DPT.
   - UPDATE on pid not in DPT → add (pid, LSN).
   - UPDATE/CLR → ATT[txn].lastLSN = LSN.
   - COMMIT → ATT[txn].status = C; ABORT → status = U.
   - END → remove from ATT.
   - End point: smallest recLSN in DPT (= redoLSN).
2. **Redo** (forward from redoLSN): *repeat history*. For each UPDATE/CLR, if pid ∈ DPT and LSN ≥ recLSN(pid) and pageLSN(pid) < LSN → reapply, set pageLSN=LSN.
3. **Undo** (backward): for each loser (status=U in ATT), follow prevLSN chain, write a **CLR** per UPDATE undone (CLR.undoNextLSN = update.prevLSN). When chain exhausted, write `END`. CLRs are redo-only (never undone).

Checkpoint

`BEGIN_CHECKPOINT` — note position. `END_CHECKPOINT` — write current ATT & DPT to log. Doesn't flush dirty pages (fuzzy). Recovery starts from BEGIN_CHECKPOINT.

DISTRIBUTED TRANSACTIONS — 2PC

Lock tables co-partitioned with data. Global deadlock: union waits-for graphs at periodic central node.

2PC protocol (assume all vote yes).

1. **Prepare phase**: Coord `flush PREPARE` (20ms log) → broadcast PREPARE → each Part receives, `flush PREPARE` → sends YES.
2. Coord receives all YES → `flush COMMIT` → broadcasts COMMIT.
3. Each Part receives COMMIT → `flush COMMIT` → sends ACK.
4. Coord receives all ACKs → `flush END` (often unlogged: presumed-commit).

Timing problem (exam).

1. Identify slowest Part round-trip: $\max_i$ 2·rtt(coord, i).
2. Total = $\log_{\text{coord-prepare}}$ + max{rtt + $\log_{\text{part-prepare}}$ + rtt + $\log_{\text{coord-commit}}$… } — work it through participant by participant.
3. Coord finishes when last ACK arrives + flush END.
4. **Crash of Part during phase 2**: coord must wait until Part recovers + redoes commit → ACK. Coord can finish at max(orig finish, recovery time + ack rtt).
5. **Crash of Coord**: on recovery reads log; if COMMIT in log → rerun phase 2; if only PREPARE → can presumed-abort or rerun PREPARE depending on protocol.

Recovery Cases

Node	Log state	Action
Coord	commit + end	Nothing (txn done)
Coord	commit only	Re-broadcast COMMIT (phase 2)
Coord	prepare/none	Presumed abort
Part	prepare + commit	Re-send ACK
Part	prepare only	Inquire coord (blocking!)
Part	none	Presumed abort

2PC is **blocking** on single failure (Part stuck after prepare). Paxos Commit / 3PC address this.

SORT-MERGE JOIN — ALGORITHM

Equi-join R(A,..) $\bowtie_A$ S(A,..). Cost: sort(R)+sort(S)+M+N (no dups on join key); refined to **3(M+N)** if Pass-0 produces ≤ B−1 runs total.

```
sort R on A; sort S on A i ← 0; j ← 0 while i < |R|
and j < |S|: if R[i].A < S[j].A: i++ elif R[i].A >
S[j].A: j++ else: // equal key js ← j while j < |S|
and S[j].A == R[i].A: output ⟨R[i], S[j]⟩ j++ j ← js //
rewind for next R i++ if i < |R| and R[i].A ==
R[i-1].A: continue else: while j < |S| and S[j].A ==
R[i-1].A: j++
```

Outer / Anti variants

- **Left outer**: when R[i].A < S[j].A, emit ⟨R[i], NULL⟩ before i++.
- **Full outer**: also emit ⟨NULL, S[j]⟩ when R[i].A > S[j].A.
- **Left anti**: emit R[i] only when R[i].A < S[j].A; never emit on equal.

EXTERNAL MERGE SORT — ALGORITHM

N pages, B buffer pages.

```
// Pass 0 — create runs for chunk in read(file, B
pages each): sort chunk in memory; write as run //
Passes 1+ — merge B-1 runs at a time while #runs > 1:
pick B-1 runs; open one input page per run + 1 output
repeat: from those B-1 input pages, pick smallest
record write to output; advance the input it came from
refill an input page when exhausted until all B-1 runs
consumed flush output as a new (longer) run
```

Passes = $1+\lceil \log_{B-1} \lceil N/B \rceil \rceil$. I/O = 2N·passes. **Blocked I/O**: read/write b pages at a time; merges $\lfloor (B-b)/b \rfloor$ runs/pass. Refined SMJ: skip the *last* merge of each sort and merge R-runs and S-runs together while joining → 3(M+N).

BLOCK NLJ — ALGORITHM

R outer (M pages), S inner (N pages), B buffers.

```
for each chunk of B-2 pages of R: load chunk into
memory hash/sorted struct for each page p of S: for
each tuple t in p: for each match m in chunk where
m.A==t.A: output ⟨m, t⟩
```

I/O = M + $\lceil M/(B-2) \rceil$·N. Pick smaller relation as outer.

3-way BNLJ (R, S, T same key)

Outer R uses P pages; load it once. Allocate $\lfloor (B-1-P)/2 \rfloor$ inner pages each to S and T. For each outer chunk, scan S and T fully. Cost ≈ N + $\lceil N/P \rceil$·N + $\lceil N/P \rceil \cdot \lceil P/(B-1-P) \rceil$·N. Optimum P balances inner rescans.

GRACE HASH JOIN — ALGORITHM

```
// Phase 1 — partition with h_p for r in R: write r to
bucket h_p(r.A) of R-side for s in S: write s to
bucket h_p(s.A) of S-side // Phase 2 — per-bucket join
with h_r ≠ h_p for each bucket i: build hash table of
R-bucket[i] using h_r for s in S-bucket[i]: for r in
HT[h_r(s.A)] where r.A==s.A: output ⟨r, s⟩ if a bucket
overflows memory → recurse with h_r as new h_p
```

Cost = 3(M+N) (read+write each side once for partitioning, then read once for build/probe). **Hash full outer**: mark matched HT entries during probe; emit unmatched HT entries with NULL after; emit unmatched probe rows with NULL as you go.

ARIES — WORKED LOG WALKTHROUGH

Pages flushed up to LSN 50; log starts at 60.

LSN	T	Type	Page	prev
60	T1	BEGIN	—	nil
70	T1	UPD	P2	60
80	T2	UPD	P3	40
90/100	—	BEGIN/END_CKPT	—	—
110	T2	UPD	P1	80
120	T1	UPD	P3	70
130	T3	BEGIN	—	nil
140	T3	UPD	P1	130
150	T1	ABORT	—	120
160	T1	CLR(120)	P3	150
170	T2	COMMIT	—	110

END_CHECKPOINT (LSN 100)

ATT: {T1:70, T2:80}. **DPT**: {P2→70, P3→80}.

After Analysis

ATT: {T1:160 (status=U after 150), T2:170 (status=C), T3:140 (U)}. **DPT**: {P2→70, P3→80, P1→110}.

Redo from minRecLSN = 70

Replay every UPDATE/CLR with LSN ≥ recLSN(page) and pageLSN < LSN.

Undo (losers = T1, T3, in reverse LSN order)

ToUndo = {160, 140}. T1's chain at 160 is a CLR — skip; follow undoNextLSN=70 → undo 70: write CLR(70, undoNext=nil), then write END for T1. T3's chain at 140 → undo: write CLR(140, undoNext=nil), then END for T3. Drop T2 (committed). Final new log: CLR(70), END(T1), CLR(140), END(T3).

2PC — WORKED TIMING

5 nodes; coord = 5; rtt 5↔1=550, 5↔2=500, 5↔3=600, 5↔4=520. Each log flush = 20ms.

1. **t=0**: coord flushes PREPARE → done at **20**; broadcasts.
2. Slowest part is N3 (600ms each way). N3 receives at 620, flushes PREPARE 620→640, sends YES at 640, arrives at coord at 1240.
3. Coord has all YES at **1240**; flushes COMMIT 1240→1260; broadcasts.
4. N3 receives COMMIT at 1860, flushes 1860→1880, sends ACK at 1880, arrives coord at **2480**.
5. Coord flushes END 2480→**2500**. **Total = 2500ms.**

Crash of N3 at 1800, recovery 2000: at recovery N3 sees PREPARE only → asks coord; coord still has COMMIT in log; resends COMMIT to N3, which flushes + ACKs. Coord finishes at 2000 + 600 (deliver) + 20 (flush at N3) + 600 (ACK back) + 20 (END) = **3240ms**.

Crash of coord: on recovery reads log; if COMMIT present → rerun phase 2 (re-broadcast); if only PREPARE → can presumed-abort (or rerun PREPARE). Participants stuck in PREPARE state must wait → 2PC is **blocking**.

B+ TREE SPLIT — WORKED EXAMPLE

Order d=2, "new right gets larger half", insert into a full leaf splits.

Insert 30 into leaf [18,27,29] of a tree where parent has separators [9,18,32,45]. Leaf overflows → new right leaf [29,30] (larger half), left keeps [18,27]. **Copy up** 29 to parent. If parent was full, parent splits: **push up** middle key (one position higher). For order d=2 inner node holds 2..4 keys; splitting 5 keys → keep 2 left, push middle, give 2 to right.

Min/Max bounds (order d, height h)

Min entries (h≥1): root has 1 key → 2 children → each minimal subtree of height h−1 has $(d+1)^{h-1}$·d entries → **$(d+1)^h$·d / (d+1)·… ≈ 2·$(d+1)^{h-1}$·d**. Max: full tree = $(2d+1)^h$·2d.

2024 Q3 example (d=3): smallest N forcing h=2 = 2·(2d+1)·d+1 = 2·7·3+1 = **43**. Largest N at h=2 = $(2d+1)^2$·2d = 49·6 = **294**.

CONFLICT GRAPH — WORKED EXAMPLE

S = ⟨R_3(B), R_1(A), W_1(B), R_2(B), R_2(A), W_3(A)⟩

1. List conflicts (same obj, ≥1 W, different txn):
   - R_3(B) — W_1(B): edge $T_3 \to T_1$.
   - W_1(B) — R_2(B): edge $T_1 \to T_2$.
   - R_2(A) — W_3(A): edge $T_2 \to T_3$.
   - R_1(A) — W_3(A): edge $T_1 \to T_3$.
2. Graph: $T_1 \to T_2 \to T_3 \to T_1$ — **cycle** ⇒ not conflict-serializable.
3. Remove R_2(B) (or W_3(A)) breaks the cycle. Topological sort of resulting DAG gives a serial equivalent.

PARALLEL & DISTRIBUTED DBMS

Arch	Shares	Notes
Shared Memory	Mem+Disk	Easy SW, costly HW, limited scale
Shared Disk	Disk	Middle ground; consistency easier
Shared Nothing	—	Cheap, scales, harder SW

Parallelism

- **Inter-query**: throughput.
- **Intra-query inter-op**: pipeline + bushy.
- **Intra-query intra-op**: data partition + per-partition op.

Partitioning

- **Round-robin**: even load, no locality.
- **Hash(A)**: equality lookup on A, equi-join on A, group-by A — local.
- **Range(A)**: range queries + above.

Distributed query plan (decision tree).

1. Selection on partition key → only relevant nodes; else broadcast or all-nodes.
2. GROUP BY g — if partitioned on g: local agg, no shuffle. Else: local partial agg → reshuffle by g → final agg.
3. MAX/SUM/COUNT: *distributive* → local + final aggregate.
4. Join R$\bowtie_A$S: see scenarios below.

Distributed Join Scenarios

1. One side replicated everywhere → local join.
2. Both partitioned on join key → local join.
3. Different keys, one side small → broadcast small side.
4. Different keys, both large → reshuffle both on join key.

Parallel Hash Join Cost

Round-robin start, k machines, R+S total: shuffle reads/writes = 2(M+N)/k per machine; rebuild = (M+N)/k. Total cross-network ≈ (M+N)·page_size·(k−1)/k. With combined partition+phase1: 3(M+N)/k disk I/Os per machine.

Parallel SMJ (range-partitioned same)

Local sort + merge: per machine 3·(M+N)/k if refined SMJ applies.

NOSQL

Web-scale needs horizontal scale; 2PC blocking; strong consistency expensive.

CAP & BASE

CAP: Consistency · Availability · Partition tolerance — pick 2.
BASE: Basically Available, Soft state, Eventually consistent.

Type	Model	Eg
Key-Value	(k,v); v opaque	Redis, DynamoDB
Document	JSON/BSON queryable	Mongo, CouchDB
Column-Family	Wide rows / families	Cassandra, HBase
Graph	Nodes & edges	Neo4j, Neptune

SQL: ACID, joins, stable schema. **NoSQL**: scale, throughput, flexible schema. Lines blur (PG JSON, Mongo ACID).

EXTENDIBLE HASHING — WORKED EXAMPLE

Bucket size 2, start global depth g=1, dir [0,1].

Insert h(k): take last g bits to find dir slot, follow ptr to bucket. If bucket full and local d<g: split bucket only (rehash entries by next bit), local d++. If d=g: **double directory** (g++); each old slot duplicated; split overflow bucket; rehash.

Search / Insert

Always **1 I/O** for the bucket (directory in memory). On split: read bucket + write 2 new buckets. Directory doubling is in-memory pointer copy.

Static hash skew (2021 Q3c)

Uniform h_p over [0,19], 100·100=10000 records → 500/bucket = 5 pages → I/O ∈ [1,5], expected ≈ 3. Skewed h_d(k)=#digits in [0,9999] → 4 buckets: 1-digit:10, 2:90, 3:900, 4:9000 records. Pages per bucket ≈ $\lceil n/100 \rceil$. Min I/O=1; Max I/O=$\lceil 9000/100 \rceil$/2=45; Expected = Σ (n_i/N)·(½ chain length).

OPTIMISTIC CC & VIEW VS CONFLICT

OCC — 3 Phases

Read (private workspace, no locks) → **Validate** (check no R/W or W/W conflict with overlapping committed txns) → **Write** (apply). Great when conflicts rare; aborts expensive under contention.

View ⊋ Conflict — example

S = W_1(A), W_2(A), R_3(A), W_3(A). All initial & final writes preserved → view-equivalent to $T_1T_2T_3$. But blind writes can be reordered without conflict-equivalence — gives a view-ser but not conflict-ser schedule.

KEY FORMULAS — QUICK REFERENCE

```
Ext sort: passes = 1+log_{B-1}⌈N/B⌉; cost = 2N · passes
Block NLJ: M + ⌈M/(B-2)⌉·N
Grace Hash / refined SMJ: 3(M+N)
B+ tree height ≈ log_F(N), F ≈ 133 typical
B+ min entries (h≥1): (d+1)^h·d
B+ max entries (h): (2d+1)^h·2d
Ext hash: max table = B(B-1); need B>√(f·N)
Join size: |R|·|S|/max{NKeys(R.A),NKeys(S.A)}
FK join: |out| = |many-side|
```